```
(V’,E’)=({2,3}, E-{1}), ({1,3}, E-{2}), ({1,2}, E-{3})
```

Each of those graphs will in turn produce two calls:

```
({3}, E-{1,2}), ({2}, E-{1,3}),
({3}, E-{1,2}), ({1}, E-{2,3}),
({2}, E-{1,3}), ({1}, E-{2,3})
```

As you can see, in the second round of calls each graph appears twice. If we also consider the recursion on the edges, it gets much worse.

Usually this kind of expansion ends up in a memory crash long before the recursion gets closer to the base cases.

The most common strategy to cope with this is to avoid the duplicates by either:

- Define a better recursion that avoids the duplicates (not always possible);
- Prune the search tree avoiding duplicated work (*branch and bound* algorithms);
- Compute and store the results for the smaller cases, and read them when needed while computing larger problems (*dynamic programming* algorithms).

For this problem, we can reasonably go for the third option, and use *memoization*[15] to provide a sort of cache of the results of the algorithm for smaller problems.

This will give us *some* improvement but, as we’ll see, the most efficient algorithms developed for planarity testing,